G-ThinkerCG: A Unified CPU-GPU Framework for Scalable Subgraph Finding

Paper ID: #94, Paper Type: Regular

Abstract

Given a large graph G, *subgraph finding* refers to a general category of problems that find all subgraphs of G meeting problem-specific criteria. Subgraph finding is a fundamental graph analytics primitive with applications in social, biological, and knowledge graphs. Despite extensive prior work on CPU- and GPU-based solutions, efficiently supporting subgraph finding on heterogeneous CPU-GPU platforms remains challenging due to irregular search spaces, severe load imbalance, and stringent GPU memory constraints.

We present G-ThinkerCG, a unified subgraph-centric framework that enables scalable and efficient subgraph finding through coordinated CPU-GPU co-processing. G-ThinkerCG is built on a unified subgraph-task abstraction that bridges the fundamentally different execution models of recursive CPU-based search and massively parallel GPU-based exploration. On the GPU, G-ThinkerCG introduces the first practical BFS-DFS hybrid subgraph extension strategy that bounds device memory usage without requiring prior knowledge of search depth. A lightweight host-side buffering mechanism transparently spills and resumes subgraph tasks when GPU memory becomes insufficient, ensuring correctness and scalability for deep and irregular searches. On the CPU, recursive DFS efficiently handles deep and straggler-heavy tasks, while dynamic bidirectional work stealing balances workloads across CPU threads and GPU warps. We implement representative subgraph finding applications on G-ThinkerCG and evaluate it on large real-world graphs. Experimental results show that hybrid CPU-GPU execution consistently outperforms CPU-only and GPU-only baselines and scales more robustly than prior state-of-the-art systems.

1 Introduction

Given a large graph $G = (V, E)$, subgraph finding (SF) enumerates or counts subgraphs of G that satisfy application-specific conditions. SF is a fundamental primitive in graph analytics with applications in social, biological, and knowledge graphs [**?**]. The search space for SF is combinatorial and typically explored by recursive backtracking over a redundancy-free subgraph enumeration tree; practical algorithms, therefore, rely on aggressive pruning and careful scheduling to make the search tractable.

GPU- and CPU-based solutions to SF each exploit different strengths of modern hardware. Existing GPU solutions typically perform *subgraph extension* over the subgraph enumeration tree using either breadth-first search (BFS) or depth-first search (DFS) strategies. BFS-style subgraph extensions enable coalesced memory access and hence high GPU throughput, but require materializing all intermediate subgraphs at each level, making them vulnerable to device out-of-memory (OOM) errors. DFS-style approaches [**? ?**] are more memory-efficient, as each thread block or warp explores a subgraph enumeration subtree using a compact stack; however, they usually require preallocated stack space in device memory and are therefore practical mainly when the search depth is known or bounded. Hybrid BFS-DFS strategies have been explored in CPU systems to balance memory usage and parallelism by bounding per-level subgraph extension [**?**]; however, directly adopting such strategies on GPUs is non-trivial due to limited device memory and the unknown extension depth of many SF problems.

At the same time, SF workloads are highly skewed and irregular. Real-world graphs commonly follow power-law degree distributions: most initial vertices spawn shallow enumeration trees (high parallelism), while a small fraction lead to very deep, irregular searches. This asymmetry suggests a natural division of labor: GPUs are well suited to process wide, shallow levels with massive parallelism, whereas CPUs are better at handling deep, straggler-prone tasks via recursive DFS. Despite this apparent complementarity, effective CPU-GPU co-processing for exact subgraph finding remains largely unexplored: the two processing contexts employ fundamentally different execution models and scheduling mechanisms, and naïvely combining them can easily lead to unbalanced workloads or catastrophic memory usage on the device.

The fundamental difficulty in designing a unified CPU-GPU system for SF is therefore a *model mismatch*. CPU-side systems expose recursive, task-centric programming and dynamic task decomposition (e.g., timeout-based splitting of long-running DFS tasks) [**? ? ?**], while GPU-side systems operate on level-wise buffers, rely on bulk extensions for throughput, and have severely constrained device memory. Bridging these models requires (i) a unifying abstraction that maps naturally to both recursive CPU execution and massively parallel GPU kernels, (ii) mechanisms to bound and transparently manage GPU memory usage during deep searches, and (iii) dynamic scheduling to move work between processors so that each executes the tasks it handles best.

To address these challenges, we present **G-ThinkerCG**, a unified subgraph-centric framework for scalable subgraph finding on heterogeneous CPU-GPU platforms. G-ThinkerCG is built around a single *subgraph-task* abstraction that treats a subgraph together with its enumeration subtree as the smallest schedulable unit; this abstraction harmonizes recursive CPU implementations and GPU-level extensions and enables bidirectional task stealing between CPU threads and GPU warps. On the GPU side, G-ThinkerCG adopts a practical BFS-DFS hybrid extension strategy that bounds per-level expansion and thus avoids unbounded device memory growth without requiring prior knowledge of maximum extension depth. To handle the cases when device memory is exhausted, we introduce a lightweight host-side buffering mechanism that transparently spills and resumes subgraph tasks. Together with a scheduling policy that routes shallow, wide tasks to the GPU and deep, irregular tasks to the CPU, these mechanisms enable efficient, memory-bounded co-processing.

Our contributions are summarized as follows:

- We present **G-ThinkerCG**, a general-purpose unified CPU-GPU framework that enables efficient co-processing of irregular subgraph-finding workloads on heterogeneous single-node platforms.

- We design a practical BFS-DFS hybrid subgraph extension strategy for GPUs that bounds device memory usage without requiring prior knowledge of search depth.
- We introduce a lightweight host-side subgraph buffering mechanism that transparently spills GPU tasks to host memory and resumes them, ensuring correctness and scalability for deep and irregular searches.
- We propose a unified *subgraph-task* abstraction and associated scheduling policies that bridge CPU and GPU programming models and support dynamic bidirectional work stealing for load balance.
- We implement representative SF applications on G-ThinkerCG and provide extensive experiments on large real-world graphs demonstrating that hybrid CPU-GPU execution consistently outperforms CPU-only and GPU-only baselines and scales more robustly than prior systems.

The rest of the paper is organized as follows. Section **??** reviews preliminaries and related work. Section **??** presents our hybrid BFS-DFS scheme on the GPU, and Section **??** describes the computation model, programming interface, and system design of G-ThinkerCG. Section **??** reports experimental results, and Section **??** concludes and discusses future work.

2 Preliminaries and Related Work

This section reviews the execution model of GPUs and the subgraph finding (SF) abstraction relevant to our system design, followed by a discussion of related work on parallel SF systems.

2.1 Preliminaries

GPU Execution Model. In CUDA, GPU kernels are executed by many threads organized into blocks, which are further divided into warps of 32 threads. A warp is the smallest scheduling unit and executes instructions in a lockstep manner; divergent control flow within a warp is serialized. Due to the GPU memory hierarchy and warp-level execution model, achieving coalesced memory access is critical for performance.

In GPU-based subgraph finding systems, warps typically process adjacency lists or vertex sets collaboratively to ensure coalesced access. GPU threads cannot directly access host memory; data must reside in device memory unless managed through CUDA Unified Memory, which provides a unified address space and transparently migrates pages between host and device as needed. These characteristics strongly influence how subgraph enumeration and task scheduling are implemented on GPUs.

Subgraph Finding (SF). Most SF algorithms perform recursive backtracking over a *subgraph enumeration tree*, where each node represents a subgraph instance and its descendants correspond to all valid extensions of that subgraph. The enumeration tree is redundancy-free: each subgraph is generated exactly once. Due to the power-law degree distribution of real-world graphs, the fanout and depth of the enumeration tree vary significantly across nodes, leading to highly irregular workloads.

Figure **??** illustrates a simple example of a subgraph enumeration tree. This enumeration model underlies a wide range of SF problems, including maximal cliques [**? ?**], maximum clique [**? ?**], maximum k-plex [**? ? ?**], maximal k-plexes [**? ? ?**], maximum quasi-clique [**?**], maximal quasi-cliques [**? ? ?**], and size-bounded community [**? ?**].

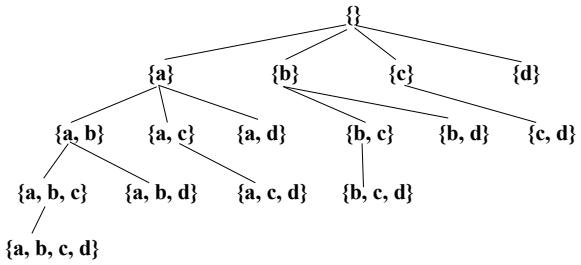


Figure 1: Set-Enumeration Tree

Another representative SF problem is subgraph matching (SM), which enumerates all subgraphs of a data graph G that are isomorphic to a query graph G_q. Many SM algorithms follow Ullmann's backtracking framework [**?**], which recursively extends partial mappings while pruning infeasible candidates. This process also corresponds to a search over a subgraph enumeration tree. We provide further details in Appendix **??** [**?**].

Subgraph Tasks. For a node S in the subgraph enumeration tree, we refer to the work of examining S and exploring all valid extensions rooted at S as a *subgraph task*, denoted by T_S. Each subgraph task corresponds to a subtree of the enumeration tree and forms a natural unit for structuring and scheduling work in parallel SF systems.

2.2 Related Work

Vertex-Centric vs. Subgraph-Centric Models. Vertex-centric systems such as Pregel [**?**] and its variants [**? ? ? ?**] rely on graph partitioning and iterative message passing. GPU-accelerated vertex-centric frameworks [**? ? ? ? ? ? ? ?**] primarily target low-complexity iterative algorithms such as BFS and PageRank [**?**]. In contrast, SF workloads are recursive and enumeration-based, motivating subgraph-centric execution models that operate on subgraph instances rather than individual vertices.

Subgraph-Centric Systems for SF. General-purpose SF systems such as Arabesque [**?**], RStream [**?**], Fractal [**?**], Pangolin [**?**], and GAMMA [**?**] adopt an extend-and-filter programming model to incrementally grow subgraphs. Among these, Pangolin and GAMMA support GPU execution. Pangolin relies on BFS-style subgraph extensions and may encounter device memory exhaustion on large graphs, while GAMMA supports out-of-core processing via host memory but incurs significant overhead due to join-heavy execution.

Earlier systems such as Arabesque, RStream, and Fractal may extend subgraphs naïvely, and therefore require explicit graph isomorphism checks to eliminate redundant subgraphs that are generated multiple times. In contrast, classical backtracking-based SF algorithms are designed to enumerate a redundancy-free subgraph enumeration tree, where each subgraph instance is generated exactly once and no post hoc deduplication is needed. Pangolin allows application-specific pruning and pattern classification to reduce redundancy, and Peregrine [**?**] introduces a pattern-centric abstraction to guide exploration. However, these programming models differ fundamentally from backtracking-based SF algorithms

and often require nontrivial reformulation to recover redundancy-free enumeration behavior. In contrast, G-thinkerCG follows the backtracking-based SF paradigm and preserves redundancy-free enumeration semantics, while providing system-level support for parallel execution and heterogeneous CPU-GPU co-processing.

Backtracking-Based SF Systems. G-thinker [**?** **?** **?**] and G-Miner [**?**] are distributed systems that allow users to express SF algorithms using recursive backtracking, closely following their serial counterparts. T-thinker [**?**] and G-thinkerQ [**?**] are single-machine variants, with G-thinkerQ supporting online subgraph queries. These systems focus on CPU-based execution and do not target heterogeneous CPU-GPU platforms.

GPU-Based and Heterogeneous SF Systems. Most GPU-based SF solutions focus on tailor-made algorithms for specific problems such as maximal clique enumeration or subgraph matching. G^2-AIMD [**?**] is the closest general-purpose GPU framework for SF, adopting BFS-style subgraph extension with adaptive chunk sizing to mitigate device OOM. However, failed chunks result in wasted computation, and GPU-only execution struggles with deep and irregular enumeration trees.

Recent CPU-GPU graph processing systems [**?** **?** **?** **?**] all target vertex-centric or approximate workloads and operate under iterative or sampling-based execution models. These approaches are orthogonal to exact subgraph finding, which requires exhaustive, recursive enumeration and fine-grained task scheduling.

Discussion. As a system paper, our focus is on architectural and scheduling mechanisms for efficient subgraph finding on heterogeneous CPU-GPU platforms. We build on representative implementations from prior work to evaluate the effectiveness of our design. Application-specific GPU optimizations (e.g., [**?**]) are complementary to our approach and may further improve performance. A broader survey of SF systems is available in [**?**].

3 BFS-DFS Subgraph Extension on a GPU

This section describes the GPU-side design of G-thinkerCG, focusing on hybrid BFS-DFS subgraph extension on a single GPU. It constitutes the GPU execution component of the overall system design presented in Section **??**. While our evaluation focuses on a single-GPU configuration, the design naturally supports multiple GPUs by instantiating multiple GPU workers, without requiring changes to the core execution model.

3.1 Overview of the Hybrid BFS-DFS Extension

Our GPU program is driven by a *GPU worker*, a CPU thread that launches GPU kernels to perform subgraph computations and extensions. To build intuition for our BFS-DFS hybrid strategy, Figure **??** presents a simplified example of extending a chunk of initial subgraphs $\{A, B\}$ through levels $L_1, L_2, \ldots$. The example assumes: (i) each subgraph at L_1–L_2 expands to three children; (ii) L_3 is the last level (no further expansion); and (iii) subgraphs are logically stored in a buffer $\mathcal{B}$ that contains contiguous segments for L_1, then L_2, then $L_3, \ldots$, where each subgraph occupies a single logical position in $\mathcal{B}$ for illustration purposes, an assumption that will be relaxed by the actual buffer representation described in Section **??**.

Users provide device UDFs that are implemented as GPU kernels and are invoked by the GPU worker (a CPU thread). The runtime manages the subgraph buffers and invokes these kernels on *segments* of the subgraph buffer $\mathcal{B}$. Specifically, each kernel invocation takes a contiguous segment corresponding to the current level L_i and concurrently extends all subgraphs in that segment to generate subgraphs at level L_{i+1}.

For each application, users implement two device functions:

- `process(.)`: given a segment of parent subgraphs from L_i, computes candidate elements (vertices or edges) for extending these subgraphs. Candidate elements are written to auxiliary device buffers.
- `extend(.)`: given the candidate elements produced for the segment of L_i, generates the corresponding extended subgraphs and appends them to L_{i+1} using system-provided atomic buffer primitives.

Within a kernel invocation, multiple warps independently fetch parent subgraphs from the input segment L_i using system-provided atomic buffer primitives, and process them in parallel. Assignment of input segments to kernel invocations is performed by the GPU worker runtime, while fine-grained work distribution across warps inside a kernel is achieved via atomic operations; user code does not perform scheduling or explicit warp mapping.

The runtime exposes lightweight primitives and counters that UDF kernels may query to bound per-level expansion and to detect imminent device-buffer overflow. During execution, a UDF inspects these mechanisms (via the runtime-provided check) to decide whether a newly generated child subgraph should be appended to the in-device output buffer or routed to a host-managed spill buffer. Concretely, when a warp observes that appending a child would exceed device buffer capacity, the warp appends that child directly into the host-managed buffer (which is accessible to the device via the CUDA managed allocation) rather than writing it into the device output segment L_{i+1}.

At each level we also bound device memory usage by limiting the number of newly appended subgraphs placed into L_{i+1} to at most η, where η is chosen substantially larger than the number of warps to allow GPU saturation while keeping in-device memory bounded. When L_{i+1} reaches this limit, the kernel stops extending further parents from the provided input segment L_i, and the runtime records the remaining positions in L_i (pushing the unfinished range onto a stack S_L) for later backtracking; after deeper levels are processed, the worker resumes the recorded segment of L_i as part of the backtracking process. Figure **??**(a) walks through ten steps of this mixed BFS-DFS procedure for the example chunk with $\eta = 6$.

Specifically, (1) the GPU worker initially provides the input segment corresponding to level L_1, which contains the chunk $\{A, B\}$, to a kernel invocation. (2) The kernel first extends subgraph A, appending its three children C, D, E to the buffer segment for L_2. (3) The kernel then extends subgraph B, appending its three children F, G, H to L_2, completing the initial population of level L_2. At this point, the runtime proceeds to process the next level.

(4) The next kernel invocation processes the segment corresponding to L_2, and subgraph C is extended first, generating I, J, K in L_3. (5) Subgraph D is then extended, generating L, M, N in L_3. Since $\eta = 6$ subgraphs have now been appended to the in-device buffer for L_3, the kernel stops extending further subgraphs from L_2. Accordingly, the runtime records the remaining unprocessed portion

of the L_2 segment by pushing the range $[4, 8)$ onto the stack S_L for later backtracking.

(6) The kernel then processes the segment for L_3. Because L_3 is the last level in this example, the subgraphs I, J, K, L, M, N are fully processed without further extension. After completing level L_3, the GPU worker backtracks to resume processing the remaining unextended subgraphs in L_2. (7) Specifically, the runtime resumes from the unprocessed portion of the L_2 segment, corresponding to positions $[4, 8)$ in $\mathcal{B}$ (popped from S_L). Subgraphs E and F are then extended, appending another $\eta = 6$ subgraphs to L_3. (8) These newly generated subgraphs in L_3 are then processed. After completing this batch, the GPU worker again backtracks to level L_2.

(9) The remaining subgraphs G and H in L_2 are then extended, generating their corresponding children in L_3. (10) Finally, these extended subgraphs in L_3 are processed, completing the enumeration for the initial chunk $\{A, B\}$.

This example highlights how the hybrid BFS-DFS strategy limits per-level expansion using the bound η, temporarily defers remaining work via backtracking, and interleaves level-wise expansion with depth-first processing to balance parallelism and memory usage.

3.2 GPU Worker Backtracking

Figure **??**(b) shows the pseudocode of the GPU worker's backtracking control loop that implements the hybrid BFS-DFS extension strategy described above. Each iteration of the loop handles subgraph extension from the current level L_i to the next level L_{i+1}.

Specifically, we represent the current level L_i as a contiguous segment $\mathcal{B}[p_{rd}, p_{wr})$ in the subgraph buffer $\mathcal{B}$. The next level L_{i+1} is written starting at position p_{wr}, and its tail is tracked by a write pointer p'_{wr} that advances dynamically as child subgraphs are appended during kernel execution. The pointer p_{rd} marks the beginning of the current level, while p_{wr} marks the initial boundary between L_i and L_{i+1}.

In addition, p'_{rd} records the position of the next parent subgraph in L_i to be processed in the current iteration, i.e., subgraphs in $\mathcal{B}[p_{rd}, p'_{rd})$ have already been extended, whereas subgraphs in $\mathcal{B}[p'_{rd}, p_{wr})$ remain to be processed. These pointer variables and their evolution during execution are illustrated in Figure **??**(c)–(d).

Figure **??**(b) shows the pseudocode of the GPU worker's backtracking control loop that implements the hybrid BFS-DFS extension strategy described above. Each iteration of the loop handles subgraph extension from the current level L_i to the next level L_{i+1}. Specifically, we represent L_i as the contiguous segment $\mathcal{B}[p_{rd}, p_{wr})$ and L_{i+1} as $\mathcal{B}[p_{wr}, p'_{wr})$, where p_{rd}, p_{wr}, and p'_{wr} are positions in the subgraph buffer $\mathcal{B}$. In addition, p'_{rd} records the position of the next parent subgraph in L_i to be processed in the current iteration, i.e., subgraphs in $\mathcal{B}[p_{rd}, p'_{rd})$ have already been extended. These pointer variables and their evolution during execution are illustrated in Figure **??**(c)–(d).

At Line 2 of Figure **??**(b), the GPU worker checks whether the current level $L_i = \mathcal{B}[p_{rd}, p_{wr})$ is non-empty. If so, the worker launches kernel invocations to process this level in Lines 3–4.

The runtime supports two execution modes for a kernel invocation depending on whether the optional UDF `process(.)` is implemented or not (the default implementation is a no-op). When the `process(.)` kernel is executed, warps independently acquire

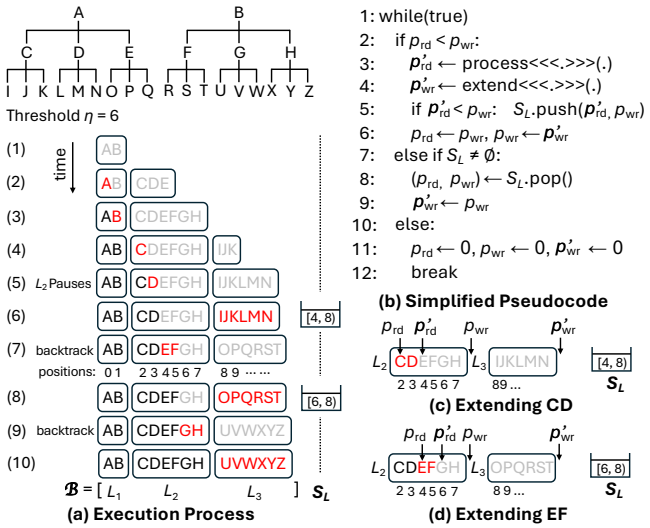


Figure 2: Example of Hybrid BFS-DFS Subgraph Extension

parent subgraphs from the input segment L_i via atomic work-acquisition and generate candidate elements for each parent into auxiliary device buffer initially preallocated by the application. Crucially, the `process(.)` phase is bounded: it stops when either all parent subgraphs in L_i have been examined or a global per-level candidate limit η has been reached (we adopt the convention that each candidate element corresponds to one potential child subgraph). When `process(.)` returns, p'_{rd} has been advanced to reflect how many parents in L_i were processed in this iteration, and the remaining parents are represented by the segment $\mathcal{B}[p'_{rd}, p_{wr})$.

Afterwards the runtime invokes the `extend(.)` kernel to materialize child subgraphs for parents in $\mathcal{B}[p_{rd}, p'_{rd})$ using the candidate elements produced by `process(.)`. In this two-kernel mode, `extend(.)` consumes candidates (each paired with its parent) and appends the resulting child subgraphs to the device output segment for L_{i+1}; because candidates are explicit, `extend(.)` can distribute work at candidate granularity and thereby achieve fine-grained load balancing across warps when parent candidate counts are skewed.

If the application chooses not to use `process(.)`, the runtime invokes `extend(.)` alone. In this single-kernel execution style, each warp acquires a parent subgraph from L_i, enumerates its candidate elements on the fly, and generates the corresponding child subgraphs directly and appends them to L_{i+1}. To maintain the same memory-control semantics as the two-kernel mode, `extend(.)` monitors the global per-level limit η during generation and stops acquiring new parents when the device-side count for L_{i+1} reaches η. Compared with the two-kernel style, this approach avoids intermediate candidate buffering but may expose less balanced workloads when different parent subgraphs produce widely varying numbers of candidates.

In both execution styles, per-level growth of the device-resident output segment for L_{i+1} is controlled in the sense that the system limits how many child subgraphs can be placed into the device buffer during one iteration.

After the `extend(.)` kernel completes, Line 5 of Figure **??**(b) records any remaining unprocessed parent subgraphs in the current level L_i. Specifically, if $p'_{rd} < p_{wr}$, the GPU worker pushes the pair

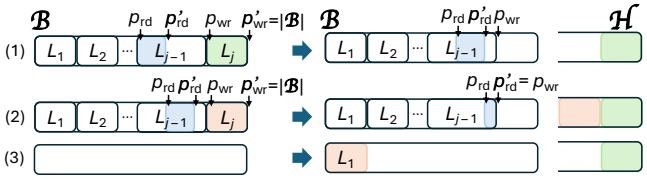


Figure 3: Example of Host Buffering of Subgraphs

of position indices $\langle p'_{rd}, p_{wr} \rangle$ onto the stack S_L, which compactly encodes the range of parent subgraphs in L_i that have not yet been extended and should be revisited later during backtracking.

At Line 6, the GPU worker advances to the next level by updating p_{rd} to the beginning of the newly generated segment for L_{i+1} and updating p_{wr} to its end. As a result, the next iteration of the backtracking loop will process level L_{i+1}. Figure **??**(c) illustrates this case, where after extending subgraphs C and D and generating a batch of child subgraphs at level L_3, the remaining segment $[4, 8)$ of level L_2 is pushed onto S_L for deferred processing.

If, at Line 2, the GPU worker finds that the current level L_i is empty, it checks whether the stack S_L is non-empty (Line 7). If so, Line 8 pops the most recently pushed segment from S_L and restores it as the new current level L_i. Line 9 then clears the output segment for L_{i+1}, ensuring that the remaining parent subgraphs in L_i can be extended into a fresh buffer segment in the subsequent iteration. Figure **??**(d) shows an example of this backtracking behavior, where after extending subgraphs E and F, the segment $[6, 8)$ is pushed onto S_L and later popped to resume extension of G, H at level L_2.

Finally, if both the current level L_i and the stack S_L are empty (Line 10), the GPU worker has finished processing the current input chunk (e.g., the chunk $\{A, B\}$ in Figure **??**(a)). In this case, Line 11 clears the buffer $\mathcal{B}$ and Line 12 terminates the backtracking loop, completing GPU-side processing for the chunk.

3.3 Stack-Based Host Buffering

In our CUDA implementation, a large contiguous space is preallocated for the subgraph buffer $\mathcal{B}$ in device memory using `cudaMalloc()`. However, when a recursion path in the subgraph enumeration tree becomes deep, it is possible that the subgraphs generated at levels $L_1, L_2, \ldots, L_j$ exhaust the available space in $\mathcal{B}$, even though further extension to levels $L_{j+1}, L_{j+2}, \ldots$ is still required. To handle such cases, we maintain a host-resident stack $\mathcal{H}$ that temporarily stores subgraphs spilled from $\mathcal{B}$. Spilled subgraphs can later be moved back from $\mathcal{H}$ to $\mathcal{B}$ for continued processing when device space becomes available.

Figure **??** illustrates this process, assuming that overflow occurs at level L_j. (1) When the `extend(.)` kernel extends parent subgraphs in level L_{j-1} to generate subgraphs in level L_j, a warp may observe that the next write position p'_{wr} reaches or exceeds $|\mathcal{B}|$. This situation can arise because multiple warps advance p'_{wr} concurrently using atomic operations. Once such a condition is detected, the kernel stops fetching additional parent subgraphs from L_{j-1} for extension. Any child subgraph whose append operation would advance the write position p'_{wr} beyond the capacity of $\mathcal{B}$ is directly pushed to the host stack $\mathcal{H}$ during kernel execution. After the kernel invocation completes, all subgraphs of level L_j that have been generated and stored in $\mathcal{B}$ are moved as a contiguous segment to the host stack $\mathcal{H}$.

(2) The GPU worker then backtracks to level L_{j-1} and resumes extending the remaining parent subgraphs that were not processed previously. If further extensions again generate subgraphs for level L_j and cause $\mathcal{B}$ to overflow, the newly generated segment of L_j is likewise moved to the top of the host stack $\mathcal{H}$. Throughout this process, we not just ensure that at most η subgraphs are generated from level L_{j-1} into L_j in each extension attempt, but also that if $\mathcal{B}$ becomes full before η subgraphs are placed, extension stops early and the generated subgraphs are spilled to $\mathcal{H}$.

(3) When the backtracking process for the current input chunk finishes, the GPU worker does not immediately fetch a new chunk of initial subgraphs (e.g., size-1 subgraphs). Instead, it first retrieves up to η subgraphs from the top of $\mathcal{H}$ (if available) and moves them into $\mathcal{B}$ as the next chunk for processing. As shown in the last row of Figure **??**, when the subgraphs fetched into L_1 of $\mathcal{B}$ are all size-j subgraphs previously spilled to $\mathcal{H}$, the subsequent extension from L_1 to L_2 effectively corresponds to extending subgraphs from level L_j to level L_{j+1}.

Finally, we implement $\mathcal{H}$ as a stack rather than a queue so that the search proceeds in a near-DFS order. Recently spilled subgraphs are processed before older ones, which helps complete deep enumeration subtrees earlier and keeps the size of $\mathcal{H}$ small in practice. This design is important for bounding host memory usage.

3.4 Subgraph Buffer Implementation

As discussed earlier, we simplified the presentation by assuming that each subgraph occupies a single logical slot in $\mathcal{B}$; in practice, a subgraph is represented by a variable-length sequence of vertex IDs. We implement both the device buffer $\mathcal{B}$ and the host stack $\mathcal{H}$ using a unified data structure, `BufferBase`, shown in Figure **??**. Using a single representation allows subgraphs to be moved between device and host transparently without invoking user-defined code.

Figure **??** illustrates the layout and operations of `BufferBase`. As indicated by ❶ and ❷, a `BufferBase` object is defined over two preallocated arrays: `vertices`, which stores subgraph contents contiguously as vertex-ID segments, and `offsets`, where each subgraph is represented by a pair `[start, end)` that indexes into `vertices`.

`BufferBase` maintains three position indices: `ohead` and `otail`, which delimit the active range of subgraphs in `offsets`, and `vtail`, which tracks the end of used space in `vertices`. When `BufferBase` represents a current level L_i in $\mathcal{B}$, the prefix before `ohead` corresponds to previously processed levels that may be revisited during backtracking; when it represents the host stack $\mathcal{H}$, `ohead` marks the bottom of the stack.

To support both user-defined UDFs and internal runtime code, we expose three lightweight device primitives that operate on this representation and are used throughout the runtime: `append_ptr(sglen)`, `next_offsets()`, and `pop_offsets()`. The rest of this subsection explains these primitives and how user kernels (or warp-level SIMT code) should call them to implement appends and reads.

Appending a newly generated subgraph is performed at warp granularity using the `append_ptr(sglen)` primitive, illustrated by ❸. Concretely, `append_ptr` is a fast, SIMT-safe reservation operation: the first lane of a warp atomically advances `otail` and `vtail` and returns the reserved start position (an index into `vertices`) to the warp. Note that in CUDA, `atomicAdd(&x, v)` returns the

old value of x before the addition; this semantics is relied upon by `append_ptr` to obtain the correct start position for the reserved segment. The primitive itself does *not* copy vertex IDs; instead, after calling `append_ptr`, all threads of the warp cooperatively write the `sglen` vertex IDs into `vertices` at the returned position using a typical warp-parallel copy (i.e., a small loop over $\lceil sglen/32 \rceil$ iterations where each lane copies a subset of the entries). The reservation also updates an internal counter used by the backtracking logic to track per-iteration progress, which UDFs may consult to bound per-level growth.

Reading a parent subgraph for extension is provided by the primitive `next_offsets()` as illustrated by ❹, which atomically advances `ohead` and returns the `[start,end)` pair for the warp to read the subgraph contents from `vertices`. Conversely, when retrieving recently spilled subgraphs from the host stack, `pop_offsets()` (as illustrated by ❺) is called to atomically decrement `otail` and return the most-recently appended `[start,end)` pair. Since `otail` must never be decremented below zero, a plain `atomicSub` cannot be used: when multiple warps concurrently subtract from `otail`, it may temporarily become negative. We, therefore, use a custom `atomicSubNonNegative` primitive (Figure ??-❻) that guarantees non-negativity; its implementation is described in Appendix ??. Both `next_offsets()` and `pop_offsets()` are SIMT-safe and are intended to be invoked at warp granularity so that the returned offsets can be used cooperatively by all lanes in the warp.

Overall, these primitives reduce the complexity of writing UDFs by abstracting concurrent buffer management into a small, warp-level interface. While atomic operations are used for bookkeeping, their overhead is known to be low in practice [??] as CUDA atomics are highly optimized, and the actual workhorse of data movement is handled cooperatively within the warp.

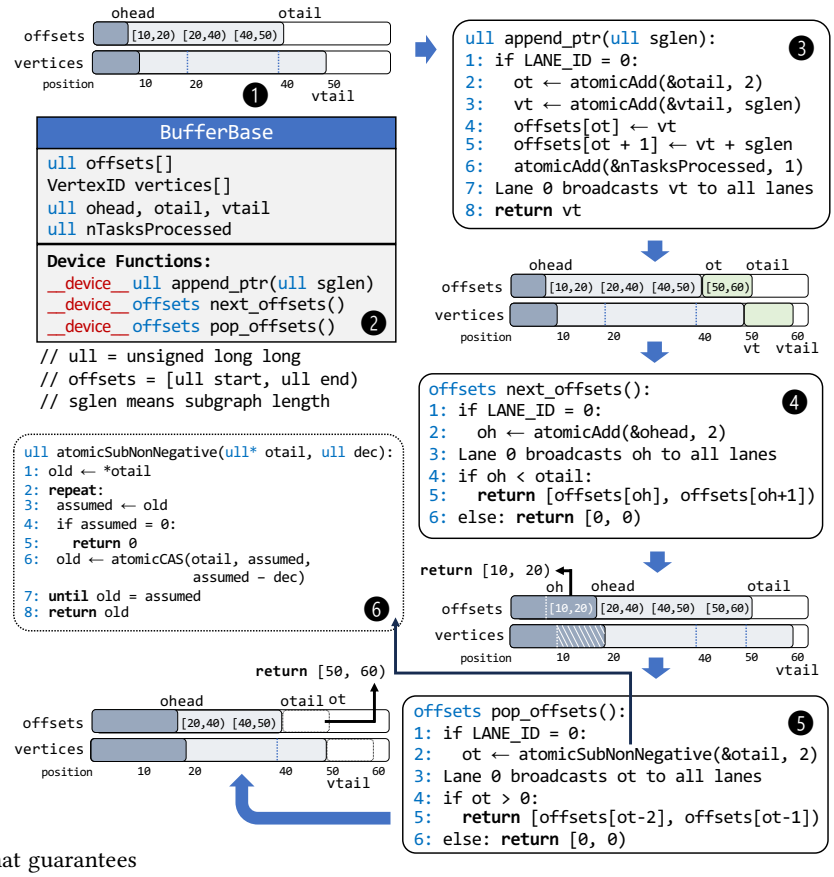


Figure 4: BufferBase and Its Operations

4 The G-ThinkerCG System

This section presents the overall computation model, programming interface, and system design of G-ThinkerCG. Building on the GPU-side hybrid BFS-DFS execution mechanisms described in the previous section, we now describe how CPU and GPU components are orchestrated within a unified system to support scalable and memory-bounded subgraph finding.

Overview. Figure ?? gives a high-level overview of the G-ThinkerCG architecture. G-ThinkerCG runs on a single multi-core machine equipped with a GPU and consists of three main components: a master thread, a group of CPU workers, and a GPU worker. The master thread is responsible for loading the input graph, creating workers, and dynamically scheduling work among them. CPU workers execute recursive, task-centric DFS to handle deep and irregular subgraph enumeration trees, while the GPU worker performs massively parallel subgraph extension using the hybrid BFS-DFS strategy described in Section ??.

Subgraph enumeration tasks are dynamically exchanged between CPU and GPU through host-resident task buffers, enabling bidirectional work stealing and adaptive load balancing. Shallow and highly parallel workloads are preferentially routed to the GPU, whereas deep or straggler-heavy tasks are processed on the CPU. This division of labor allows G-ThinkerCG to exploit the complementary strengths of CPU and GPU execution while bounding device memory usage and maintaining steady progress for highly irregular search spaces.

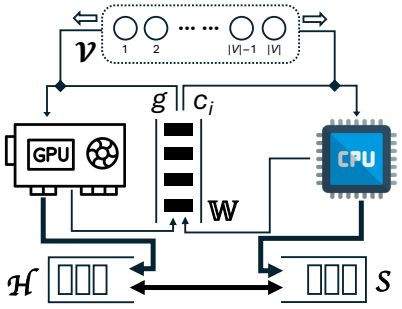


Figure 5: System Overview

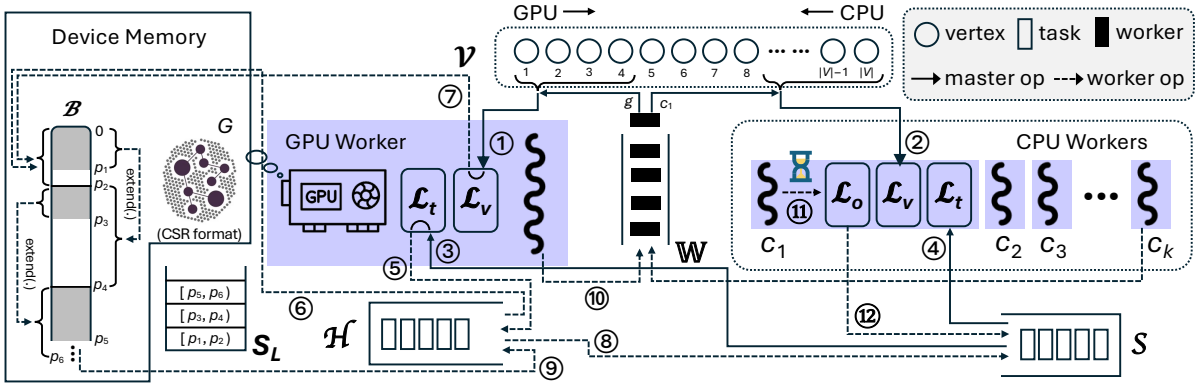


Figure 6: Detailed G-ThinkerCG System Architecture

Algorithm 1 Pseudocode of the Master Thread

```
1: Create n_c CPU workers and a GPU worker and add them to 𝕎
2: 𝒮 ← ∅, 𝒱 ← V          // note that G = (V, E)
3: while |𝕎| ≠ n_c + 1 or 𝒱 ≠ ∅ or 𝒮 ≠ ∅ do
4:     if 𝕎 = ∅ then wait to be notified by a worker
5:     if 𝒱 = ∅ and 𝒮 = ∅ then continue
6:     w ← 𝕎.DEQUEUE()
7:     if w is a GPU worker then
8:         if 𝒱 ≠ ∅ then w.ℒ_v ← 𝒱.POP_FRONT_BATCH()
9:         else w.ℒ_t ← 𝒮.POP_BATCH()
10:    else          // w is a CPU worker
11:        if 𝒮 ≠ ∅ then w.ℒ_t ← 𝒮.POP_BATCH()
12:        else w.ℒ_v ← 𝒱.POP_BACK_BATCH()
13:    notify w to wake up for processing
14: global_end_flag ← true, notify all workers (to terminate)
```

4.1 Master

The master thread is responsible for global coordination in G-ThinkerCG. It initializes the system, manages shared task queues, and dynamically assigns work to CPU workers and the GPU worker based on their availability and the current workload characteristics. Algorithm ?? provides a pseudocode summary of the master thread.

At a high level, the master maintains two global work pools: a queue $\mathcal{V}$ of initial vertex chunks and a stack $\mathcal{S}$ of spilled subgraph tasks generated during execution (see Figure ??). The tasks in $\mathcal{S}$ are generated by CPU-side timeout-based task decomposition [? ? ?]. Idle CPU and GPU workers are placed in a worker queue $\mathbb{W}$ and dequeued by the master when work becomes available.

Each worker w maintains two local input buffers to receive work assigned by the master: $\mathcal{L}_v$, which stores a batch of initial vertices drawn from $\mathcal{V}$, and $\mathcal{L}_t$, which stores a batch of subgraph tasks drawn from $\mathcal{S}$. Figure ?? further refines the high-level design in Figure ?? by illustrating these per-worker buffers and the detailed task flows among the master, CPU workers, and the GPU worker. When a worker is assigned work, the master decides whether to dispatch an initial vertex chunk from $\mathcal{V}$ or a task chunk from $\mathcal{S}$ into the corresponding local buffer, depending on the worker type and the system state.

In G-ThinkerCG, users may order vertices in $\mathcal{V}$ using a degeneracy ordering, obtained by running the peeling algorithm for k-core decomposition [?], which iteratively removes the minimum-degree vertex. This ordering tends to place vertices with smaller candidate sets earlier, and generally keeps the candidate sets of initial vertices not too large (bounded by graph degeneracy) since their lower-degree neighbors have been considered and removed in previous branches. Such an ordering follows the most-constraining-variable heuristic commonly used in constraint satisfaction problems, which tends to maximize pruning effectiveness in the search. As a result, degeneracy ordering is widely adopted as the first step in subgraph finding algorithms based on set-enumeration trees [? ?].

To illustrate by Figure ?? on Page ??, vertices under the root are ordered as $\{d\}, \{c\}, \{b\}, \{a\}$ to form $\mathcal{V}$, so that their subtrees at the front are shallow, but those at the tail are deep.

The dispatch policy of G-ThinkerCG differentiates between CPU workers and the GPU worker to exploit their complementary strengths. **When the dequeued worker is the GPU worker** and $\mathcal{V}$ is non-empty, the master assigns a vertex chunk from the *front* of $\mathcal{V}$. Such chunks typically induce shallow and highly parallel subgraph enumeration trees, which are well suited for level-wise GPU execution. This preference allows the GPU worker to efficiently exploit massive parallelism on wide frontier levels while keeping device memory usage bounded.

If $\mathcal{V}$ is empty, the GPU worker is instead assigned a batch of spilled tasks from $\mathcal{S}$ (if available). The tasks in $\mathcal{S}$ are generated by CPU-side timeout-based decomposition [? ? ?], and the GPU may process a subset of them to assist with remaining work and improve overall load balance across CPU and GPU resources.

In contrast, **when the dequeued worker is a CPU worker**, the master gives priority to spilled tasks in $\mathcal{S}$. This policy biases execution toward a near-DFS traversal order, allowing recently spilled tasks to be processed promptly and preventing $\mathcal{S}$ from growing excessively. Only when $\mathcal{S}$ is empty does the master assign a vertex chunk from the *back* of $\mathcal{V}$ to a CPU worker. Such vertex chunks typically correspond to deep or irregular enumeration subtrees that are less amenable to level-wise GPU execution and are more efficiently handled by recursive DFS on the CPU.

The master detects termination when all CPU and GPU workers are idle and placed in $\mathbb{W}$, and both $\mathcal{V}$ and $\mathcal{S}$ are empty. At this point, no further subgraph enumeration tasks remain, so the master signals all workers to terminate gracefully.

To avoid busy waiting during execution, the master and workers coordinate using lightweight condition variables, allowing idle threads to sleep until new work becomes available.

4.2 GPU Worker

We now describe the execution model of the GPU worker, which is implemented as a dedicated host-side CPU thread responsible for launching GPU kernels and managing the GPU execution buffer $\mathcal{B}$ as well as the host-resident spill stack $\mathcal{H}$. Algorithm ?? presents the control flow of the GPU worker.

As shown in Figure ??, when the master assigns subgraph tasks from the global stack $\mathcal{S}$ to the GPU worker, they are first placed into the worker's local buffer $\mathcal{L}_t$ (see arrow ③). The GPU worker then explicitly moves these tasks from $\mathcal{L}_t$ into the spill stack $\mathcal{H}$ (see arrow ⑤), from which tasks are subsequently scheduled into the GPU execution buffer $\mathcal{B}$ for processing (see arrow ⑥). In contrast, the master does not directly manipulate either $\mathcal{H}$ or $\mathcal{B}$; it only mediates task delivery between $\mathcal{S}$ and $\mathcal{L}_t$.

The intermediate buffer $\mathcal{L}_t$ serves as a handoff boundary between the master and the GPU worker, enabling the GPU worker to manage $\mathcal{H}$ and $\mathcal{B}$ without fine-grained locking. The detailed computation and memory-management logic over $\mathcal{B}$ and $\mathcal{H}$, including hybrid BFS-DFS subgraph extension, per-level bounding, and task spilling, has been described in Section ??. Here, we focus on how the GPU worker orchestrates these mechanisms at the system level.

Overview. Once activated by the master, the GPU worker executes in a self-contained manner following the control flow shown in Algorithm ??. Each activation processes the tasks currently assigned in $\mathcal{L}_v$ or $\mathcal{H}$ (containing stolen tasks obtained from $\mathcal{L}_t$) and terminates only after all such work has been exhausted. During this execution, the GPU worker repeatedly schedules tasks from the spill stack $\mathcal{H}$ or generates initial tasks from $\mathcal{L}_v$, launches GPU kernels to perform subgraph extension, and dynamically manages task spilling and backtracking. Upon completion, the GPU worker returns to idle worker queue $\mathbb{W}$ to await next scheduling decision.

GPUContext and Buffers. Figure ?? shows the `GPUContext` object maintained by the GPU worker, which encapsulates the execution buffers, auxiliary state, and UDF interfaces required for GPU-side subgraph enumeration. At its core is the subgraph execution buffer $\mathcal{B}$, logically organized as $\mathcal{B} = [L_1, L_2, \ldots, L_i, L_{i+1}]$.

Two iterators are defined over $\mathcal{B}$: `Brd`, corresponding to the current level L_i for reading subgraphs via `BufferT::next_offsets()` (recall Figure ??-❹), and `Bwr`, corresponding to the next level L_{i+1} for appending extended subgraphs via `BufferT::append_ptr()` (recall Figure ??-❸). In addition, the GPU worker manages a host-resident spill stack $\mathcal{H}$ using the same `BufferT` abstraction, which is accessed by GPU kernels through unified addressing. The array `source[]` provides a view of the vertex chunk stored in $\mathcal{L}_v$ and is used for generating initial subgraph tasks.

User-Defined Functions. `GPUContext` exposes three user-defined device functions:

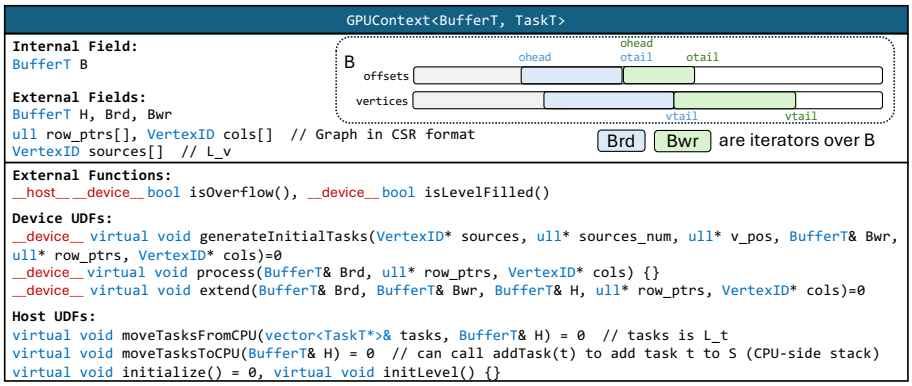


Figure 7: The GPUContext class with UDFs

- **`generateInitialTasks(.)`** generates initial subgraphs into `Bwr` (= L_1) from the vertex chunk in $\mathcal{L}_v$ (Algorithm ?? Lines 10–11, Figure ??-⑦). Since $|\mathcal{L}_v|$ may exceed the per-iteration bound η, the parameter `v_pos` (Line 5) tracks the next vertex to be processed and is forwarded across iterations of `generateInitialTasks(.)` invocations. In each invocation, up to η vertices are consumed from $\mathcal{L}_v$ to generate initial subgraphs, bounding the amount of work introduced into L_1 per iteration.
- **`process(.)`** optionally computes candidate elements for subgraphs read from `Brd` and writes them into an intermediate buffer Q, which is allocated by the host UDF `initialize()` and cleared in `initLevel()` before each iteration.
- **`extend(.)`** materializes child subgraphs by extending those read from `Brd`. Extended subgraphs are appended to `Bwr` when device memory permits, or redirected to the spill stack $\mathcal{H}$ otherwise. When `process(.)` is used, `extend(.)` consumes candidates from Q to achieve finer-grained load balancing.

Overflow Handling. To bound GPU memory usage during subgraph generation, the GPU worker uses two predicates, `isLevelFilled(.)` and `isOverflow(.)`, which are evaluated on the device and the host, respectively. During kernel execution, `isLevelFilled(.)` prevents unbounded per-level expansion by stopping further extensions when either the per-level limit η is reached (checked via `Bwr.nTasksProcessed`) or the device buffer is close to full (checked via `isOverflow(.)`). After each invocation of `extend(.)`, the GPU worker (which is a CPU thread) checks `isOverflow(.)` on the host and, if necessary, spills the newly generated output level `Bwr` to the host stack $\mathcal{H}$ for deferred processing (recall Figure ??). We conservatively trigger overflow when the utilization of `offsets[]` or `vertices[]` exceeds 95%, which tolerates concurrent appends by multiple warps while maintaining high device utilization.

GPU-CPU Task Exchange. To support bidirectional load balancing, the GPU worker may return tasks from $\mathcal{H}$ to the stack $\mathcal{S}$ when a sufficient number of CPU workers are idle (Algorithm ?? Line 9, Figure ?? arrow ⑧). Both directions of task movement, $\mathcal{L}_t \rightarrow \mathcal{H}$ (Line 4) and $\mathcal{H} \rightarrow \mathcal{S}$ (Line 8), are executed by the GPU worker.

GPU Worker Execution Flow. Algorithm ?? summarizes the execution cycle of the GPU worker. When awakened by the master (Line 2), the GPU worker processes the tasks assigned in its local

Algorithm 2 Pseudocode of the GPU Worker

```
1:  while true do
2:      wait to be notified by master
3:      if global_end_flag = true then break
4:      if L_t ≠ ∅ then H ← moveTasksFromCPU(L_t)   // ⑤
5:      v_pos ← 0    # iterating position in L_v
6:      while true do
7:          if H ≠ ∅ then
8:              pop up to η tasks from H to add to Bwr (= L_1) // ⑥
9:              if |W| > τ_c then S ← moveTasksToCPU(H)   //⑧
10:         else if v_pos < |L_v| then
11:             Bwr, v_pos ← generateInitialTasks⋘.⋙(L_v, v_pos)
    //⑦
12:         else break
13:         Brd ← Bwr (i.e., L_1) and update Bwr as L_2
14:         run the algorithm in Figure ??(b) to extend Brd = L_1
15:     L_v ← ∅    # clear the iterator for correct invocation next time
16:     append itself to W and notify the master   // ⑩
```

buffers and runs until all such work is exhausted, after which it returns to the idle worker queue $\mathbb{W}$ (Line 16). If the global termination flag is set by the master due to detection of job completion, the worker terminates immediately (Line 3).

Within each activation, the GPU worker repeatedly prioritizes tasks in the spill stack $\mathcal{H}$ for execution. If $\mathcal{H}$ is non-empty, up to η tasks are scheduled into the first level of the execution buffer $\mathcal{B}$ for GPU processing (Lines 7–8). Otherwise, up to η initial subgraph tasks are generated from the vertex chunk in $\mathcal{L}_v$ (Lines 10–11). During execution, the GPU worker dynamically manages backtracking, overflow, task spilling, and task exchange as previously described. Upon completion, the local buffer $\mathcal{L}_v$ is cleared for future use, marking the end of the current scheduling round.

4.3 CPU Workers

CPU workers are responsible for processing deep and irregular subgraph enumeration tasks using recursive, task-centric execution. In contrast to the GPU worker, which focuses on massively parallel, memory-bounded expansion, CPU workers handle workloads characterized by deep recursion, irregular control flow, or high skew. This complementary division of labor enables G-thinkerCG to efficiently support heterogeneous subgraph finding workloads.

As shown in Figure ??, each CPU worker is implemented as a host-side thread and maintains the same local input buffers $\mathcal{L}_v$ and $\mathcal{L}_t$ as the GPU worker. The master assigns work to CPU workers by placing vertex chunks into $\mathcal{L}_v$ (see arrow ②) or subgraph tasks into $\mathcal{L}_t$ (see arrow ④), after which the CPU worker processes the assigned work independently until completion.

Task execution. When processing tasks in $\mathcal{L}_t$, a CPU worker executes each task using recursive backtracking over the subgraph enumeration tree. Each task corresponds to a subtree rooted at a subgraph instance (recall Figure ??) and is processed in a depth-first manner. This execution model closely follows classical CPU-based subgraph finding algorithms [? ?] and is well suited for exploring deep search spaces that are inefficient for GPU execution.

If $\mathcal{L}_t$ is empty, the CPU worker processes vertex chunks in $\mathcal{L}_v$ (obtained from the front of $\mathcal{V}$) by spawning initial tasks from vertices and executing them locally. This policy prioritizes spilled tasks (resulted from task decomposition to be described next) over

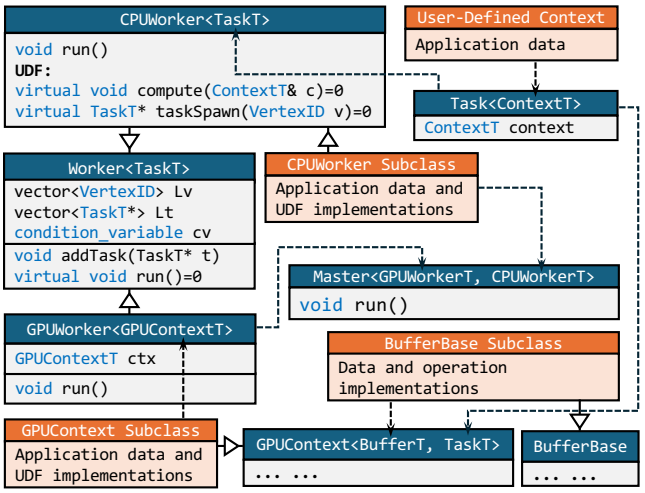


Figure 8: Programming Interface

new vertex chunks, promoting near-DFS traversal and reducing the growth of the global task stack $\mathcal{S}$.

Timeout-based task decomposition. To mitigate straggler effects caused by highly unbalanced enumeration trees, G-thinkerCG employs a timeout-based task decomposition mechanism on CPU workers. If a task executes for longer than a predefined threshold τ_{time} (0.1 seconds by default), the CPU worker suspends the task and decomposes its remaining search space into smaller sub-tasks. These sub-tasks are added back to the global task stack $\mathcal{S}$ for redistribution to other idle workers. We inherit this mechanism from prior work on task-centric subgraph finding (e.g., [? ?]) for the CPU interface of G-thinkerCG.

To reduce synchronization overhead, newly generated sub-tasks are first buffered locally and flushed to $\mathcal{S}$ in batches using a single lock operation (Figure ??, ⑪–⑫). This batching strategy minimizes contention on the shared task stack $\mathcal{S}$ while preserving responsiveness to straggler workloads.

4.4 Programming Interface

Figure ?? shows the C++ programming interface of G-thinkerCG. Classes shown in orange are user-defined and specify application-specific logic, while the remaining classes are provided by the runtime. This interface is designed to allow users to implement subgraph finding algorithms in a task-centric manner while abstracting away the complexity of CPU-GPU coordination and scheduling.

At the system level, both `GPUWorker` and `CPUWorker` inherit from a common base class `Worker`, which maintains the local input buffers $\mathcal{L}_v$ and $\mathcal{L}_t$ and a condition variable for coordination with the master. The master creates and manages all workers and invokes their execution through a unified `run()` interface.

GPU-Side Interface. On the GPU side, `GPUWorker` contains a `GPUContext` object, parameterized by a user-defined buffer type `BufferT` and a CPU-side task type `TaskT`. The `GPUContext` class encapsulates all GPU-related state, including the execution buffer $\mathcal{B}$, the spill stack $\mathcal{H}$, and the user-defined functions (UDFs) used for subgraph generation and extension. Users implement a subclass of `GPUContext<BufferT, TaskT>` to define application-specific GPU logic and pass it to `GPUWorker`.

This design avoids the use of C++ virtual functions in device code, which are not supported by CUDA, while still allowing flexible specialization through templates and user-defined subclasses. As a result, GPU kernels can directly invoke UDFs without incurring runtime polymorphism overhead.

CPU-Side Interface. On the CPU side, users define an application-specific context type `ContextT` that captures the state of a subgraph enumeration task. This context is wrapped in a task type `TaskT`, which is processed by `CPUWorker` using recursive, task-centric execution. Users provide UDFs that specify how initial tasks are spawned from vertices and how each task is processed and extended based on its context.

CPU workers execute tasks using the timeout-based decomposition mechanism described in Section **??**, allowing long-running or irregular tasks to be dynamically decomposed and redistributed.

Summary. Overall, the programming interface of G-thinkerCG enables users to express subgraph finding algorithms in a form close to their sequential or CPU-based implementations, while the runtime transparently manages heterogeneous execution, task scheduling, and CPU-GPU load balancing.

5 Experimental Evaluation

Our experiments aim to answer the following questions: (i) how G-thinkerCG performs compared with CPU-only and GPU-only solutions, (ii) how effective CPU-GPU co-processing is for irregular subgraph finding workloads, and (iii) how sensitive the system is to key scheduling parameters. Our code has been released at https://anonymous.4open.science/r/G-ThinkerCG-AFD6/.

Experimental Setup. We conduct experiments on a machine with an NVIDIA Tesla P100 GPU (16 GB memory), an Intel Xeon E5-2680 v4 CPU (14 cores, 28 threads w/ hyperthreading), and 256 GB DDR4 RAM. Unless otherwise specified, we use all CPU cores as CPU workers, and use one GPU worker. The input graphs are stored in CSR format in host memory and transferred to the GPU as required.

We run G-thinkerCG in three modes: (1) **CPU-Only** which runs with 28 CPU workers, (2) **GPU-Only** where each GPU kernel is configured with 56 blocks (since P100 has 56 streaming multiprocessors) and 1024 threads per block, and (3) **Hybrid** which runs CPU-GPU co-processing. For the device memory, $\mathcal{B}$ is allocated with all the leftover memory after storing the graph and application-specific warp buffers for keeping intermediate results. For the host memory, $\mathcal{H}$ and $\mathcal{S}$ each is allocated with half of the memory.

After completing an assigned chunk, a worker waits in the idle queue $\mathbb{W}$ for new work, incurring scheduling overhead. To amortize this cost, we use different chunk sizes for CPU and GPU workers. Specifically, CPU workers process vertex chunks of size 50, while the GPU worker uses chunks of size 500K. Smaller chunks allow CPUs to react quickly to straggler-heavy tasks, whereas larger chunks help the GPU sustain high throughput and reduce scheduling overhead. These settings work well across all experiments.

Unless otherwise specified, we use the following default parameter settings. The timeout threshold τ_{time} controls when a CPU worker splits a long-running subgraph task into smaller tasks for redistribution. We set $\tau_{\text{time}} = 0.1$ seconds, which strikes a balance between limiting task-splitting overhead and preventing straggler tasks from dominating execution. We observe stable performance within a reasonable range around this value. The CPU-GPU task exchange threshold τ_c (recall Algorithm **??** Line 9) determines when the GPU worker returns tasks from the GPU-side spill stack $\mathcal{H}$ to the CPU-side task stack $\mathcal{S}$. By default, we set $\tau_c = n_c/2$, where n_c is the number of CPU workers. This heuristic avoids excessive task thrashing between CPU and GPU while allowing timely redistribution when CPU resources are underutilized.

Applications. We evaluate G-thinkerCG using two representative subgraph finding applications: maximal clique enumeration (MCE) and subgraph matching (SM).

Maximal clique enumeration (MCE). MCE is a classical enumeration problem with highly irregular and straggler-prone search spaces. The underlying subgraph enumeration trees are typically deep and exhibit severe workload skew, making MCE challenging for both CPU-only and GPU-only systems. This application stresses the system's ability to handle deep recursion and load imbalance.

Subgraph matching (SM). SM enumerates all embeddings of a query graph in a data graph. Compared with MCE, the search depth of SM is bounded by the size of the query graph and is therefore more structured. However, SM still exhibits workload variation due to imbalanced vertex degree distribution in real power-law graphs.

These are exactly the two algorithms used in G^2-AIMD [**?**] to evaluate their GPU-only system. We closely follow [**?**] for the GPU-side application implementations, as our goal is not to introduce new application-specific optimizations, but to evaluate the effectiveness of G-thinkerCG as a general-purpose CPU-GPU unified system. Since G-thinkerQ [**?**] already provides task-centric CPU implementations for both MCE and SM, we directly build upon them for the CPU-side application implementations in G-thinkerCG.

For MCE, one key difference is that G^2-AIMD performs subgraph extension directly within `extend(.)`, which can lead to load imbalance under highly irregular search spaces. In contrast, G-thinkerCG separates candidate generation from subgraph extension, enabling finer-grained GPU tasks and better load balancing. For SM, we follow the *prefix* mode (a.k.a. super-subgraph scheme) of [**?**] for GPU execution. Our design choice avoids re-engineering application logic and ensures that G-thinkerCG inherits well-tested CPU and GPU algorithms, while extending them to a unified CPU-GPU execution setting.

Performance Comparison on Maximal Clique Enumeration (MCE). Table **??** shows the MCE running time (unit: second) on 20 large real graph datasets for the 3 modes of G-thinkerCG, and our baseline G^2-AIMD [**?**]. The Bron-Kerbosch algorithm variant with pivoting is implemented in all systems, and the graphs are all from KONECT [**?**] and Network Repository [**?**], selected to be large enough so that MCE is challenging but can be solved in reasonable time. As Table **??** shows, the maximum degree d_{max} is huge for most graphs, making MCE quite challenging.

We can see that GPU-Only beats G^2-AIMD on 15 out of the 20 graphs, indicating that our BFS-DFS hybrid strategy is superior than the chunk-adaptive BFS solution of G^2-AIMD which may incur wasted chunk computation due to OOM.

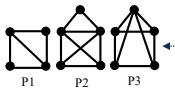
More importantly, Hybrid is consistently faster than both CPU-only and GPU-only, showing that the CPU-GPU co-processing is very effective. In particular, Gain^{H2G} shows the running time ratio for GPU-only v.s. Hybrid which is up to 9.5 (i.e., Hybrid is up to

Table 1: Performance Comparison on Maximal Clique Enumeration (MCE)

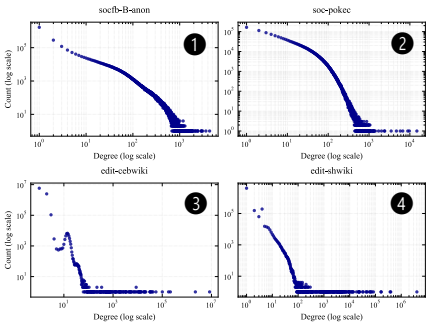
G	$\|V\|$	$\|E\|$	d_{max}	G²-AIMD	CPU-Only	GPU-Only	Hybrid	Gain^{H2G}	Gain^{H2C}
soc-sinaweibo	58.65M	522.6M	278K	108.2	205.6	150.1	109.2	1.4	1.9
trackers	27.66M	281.2M	11,571K	51.1	28.8	47.8	30.5	1.6	0.9
wikipedia_link_en	25.92M	1086M	4271K	430.5	334.2	446.4	253.0	1.8	1.3
edit-cebwiki	8.482M	23.58M	8421K	9.9	1.3	7.8	1.2	6.3	1.0
wikipedia_link_ceb	7.891M	127.8M	3636K	17.6	4.5	15.1	2.4	6.2	1.9
edit-svwiki	7.570M	43.88M	5669K	17.4	6.4	14.2	3.8	3.8	1.7
edit-enwiktionary	5.812M	54.23M	3504K	11.5	13.1	10.0	5.8	1.7	2.3
edit-zhwiki	5.005M	33.08M	1208K	6.1	7.8	5.1	3.9	1.3	2.0
edit-itwiki	4.799M	63.84M	1076K	17.8	30.9	15.0	11.8	1.3	2.6
edit-shwiki	4.579M	12.53M	4043K	5.7	1.3	4.3	1.3	3.3	1.0
edit-mgwiktionary	4.063M	17.65M	4059K	0.7	0.3	1.5	0.2	9.5	1.9
soc-livejournal	4.033M	55.86M	2651	5.3	11.6	6.8	4.3	1.6	2.7
edit-viwiki	3.496M	27.30M	2187K	3.0	1.0	2.4	0.8	3.0	1.3
edit-frwiktionary	3.411M	26.85M	2487K	4.5	1.3	3.6	1.2	3.0	1.1
wikipedia_link_sr	3.175M	206.6M	369K	1279.8	725.9	1097.6	462.7	2.4	1.6
edit-jawiki	2.981M	42.83M	385K	3.5	7.0	3.0	2.4	1.3	2.9
socfb-B-anon	2.937M	41.91M	4356	2.5	5.0	2.3	1.8	1.3	2.7
edit-plwiki	2.646M	42.43M	850K	11.9	19.6	9.9	7.6	1.3	2.6
edit-ukwiki	2.132M	17.44M	306K	1.8	2.5	1.6	1.2	1.3	2.0
soc-pokec	1.632M	44.60M	14.8K	1.7	3.2	1.7	1.0	1.7	3.3

Table 2: Performance Comparison on Subgraph Matching (SM)

(a) Query Graphs:



(b) Degree Distributions:



(c) Time:

G	Query	G²-AIMD	CPU-Only	GPU-Only	Hybrid	Gain^{H2G}	Gain^{H2C}	STMatch	EGSM	GAMMA
soc-sinaweibo	P1	35.5	OOT	58.4	59.2	1.0	–	Error	OOM	OOM
	P2	292.1	OOT	327.7	322.1	1.0	–	Error	OOM	OOM
	P3	470.7	OOT	431.6	436.7	1.0	–	Error	OOM	OOM
edit-cebwiki ❸	P1	222.2	65.7	5.3*	5.3*	1.0	12.50	633.9	35.7	312.3
	P2	361.0	OOT	26.0*	26.0*	1.0	–	OOT	71.6	OOM
	P3	OOT	OOT	115.2	115.0	1.0	–	OOT	Error	OOM
edit-shwiki ❹	P1	439.2	24.3	4.5*	4.5*	1.0	5.40	106.7	65.2	152.5
	P2	728.1	18.9	0.9*	0.9*	1.0	21.52	151.8	9.7	214.1
	P3	OOT	OOT	1165.2	1162.8	1.0	–	198.4	OOM	OOM
edit-mgwiktionary	P1	0.6	141.5	0.3	0.7	0.5	204.74	OOT	11.8	759.9
	P2	0.8	30.2	0.3	0.5	0.6	64.02	710.1	4.6	549.9
	P3	10.0	213.3	0.5	6.9	0.1	30.92	OOT	13.1	554.4
soc-livejournal	P1	2.2	8.3	2.0	2.3	0.9	3.59	26.9	OOM	16.5
	P2	61.3	476.8	51.7	49.9	1.0	9.56	OOT	OOM	OOM
	P3	102.3	767.5	94.6	94.5	1.0	8.13	OOT	OOM	OOM
socfb-B-anon ❶	P1	1.7	5.7	1.5	1.7	0.9	3.28	7.4	22.6	14.5
	P2	14.1	34.2	10.2	10.2	1.0	3.35	32.9	OOM	OOM
	P3	31.5	157.2	22.2	22.6	1.0	6.96	31.7	OOT	OOM
soc-pokec ❷	P1	2.0	2.8	1.8	1.0	1.7	2.68	13.1	12.1	7.8
	P2	6.5	9.5	6.0	4.0	1.5	2.38	9.1	196.7	24.2
	P3	13.8	53.2	7.7	7.9	1.0	6.70	11.2	OOM	106.6

9.5× faster than GPU-only), and Gain^{H2C} shows the running time ratio for CPU-only v.s. Hybrid which is up to 3.3.

Performance Comparison on Subgraph Matching (SM). Table **??**(c) shows the SM running time (unit: second) on 7 of the 20 graphs used in Table **??** for 3 representative query graphs shown in Table **??**(a). The programs run out of our maximum allowed time threshold of 1800 seconds (denoted by OOT) on the other 13 graphs.

Note that while we use the *prefix* mode of [**?**] by default which lets a warp extend a subgraph g with all its candidates so that the set intersection results on g can be shared, this leads to load balancing problems on *edit-cebwiki* and *edit-shwiki* where many vertices have very high fanouts (see Table **??**(b)-❸❹) – unlike the other power-law graphs (see Table **??**(b)-❶❷) – which leads to extreme load imbalance. Therefore, we switch to the *expand* mode of [**?**] (where each warp extends with one candidate) on both graphs for P1 and P2 which reduces the running time significantly. These numbers have been highlighted with asterisks in Table **??**(c).

We can see that GPU-Only is faster than G²-AIMD in most cases. However, Hybrid does not improve GPU-Only in most cases other than *soc-pokec*. This is because unlike in MCE, all enumeration trees are of the same height (equal to the query subgraph size) in SM, so CPU is not as complementary; tasks assigned for CPU processing lag far behind GPU processing, bottlenecking Hybrid's throughput. Based on this observation, we recommend not to enable CPU processing in G-ThinkerCG for the SM application.

We also include the latest GPU solutions to SM for comparison, STMatch [**?**], EGSM [**?**] and GAMMA [**?**], the results of which are shown in the last three columns of Table **??**(c). We can see that all three systems are much slower than GPU-Only and Hybrid, which shows the superiority of our BFS-DFS hybrid strategy for subgraph extension on a GPU.

System Parameter Study. Due to space constraints, we report a detailed system parameter study in Appendix **??**. Overall, the default parameter settings provide robust performance in general.

6 Conclusions and Future Work

We presented G-thinkerCG, a general-purpose programming framework for subgraph finding (SF) that enables unified CPU-GPU co-processing. G-thinkerCG combines a task-centric programming model with a BFS-DFS hybrid execution scheme on the GPU, allowing it to effectively handle highly irregular and straggler-prone SF workloads. Experimental results demonstrate that G-thinkerCG outperforms CPU-only and GPU-only approaches and achieves efficient CPU-GPU collaboration.

As future work, we plan to extend G-thinkerCG to support additional SF applications such as quasi-cliques and k-plexes, as well as to explore multi-GPU and distributed deployments.

References

[1] Network Datasets from KONECT. http://konect.cc/networks/.

[2] Network repository. https://networkrepository.com.

[3] Online technical report. https://anonymous.4open.science/r/G-ThinkerCG-AFD6/technical-report.pdf.

[4] A. Ahmad, D. Yan, X. Chen, L. Yuan, Q. Zhang, and S. Adhikari. Maximum k-plex finding: Choices of pruning techniques matter! *Proc. VLDB Endow.*, 18(9):2928–2940, 2025.

[5] A. Ahmad, L. Yuan, D. Yan, G. Guo, J. Chen, and C. Zhang. Accelerating k-core decomposition by a gpu. In *ICDE*. IEEE, 2023.

[6] M. Almasri, Y. Chang, I. E. Hajj, R. Nagi, J. Xiong, and W. W. Hwu. Parallelizing maximal clique enumeration on gpus. In *PACT*, pages 162–175. IEEE, 2023.

[7] V. Batagelj and M. Zaversnik. An o(m) algorithm for cores decomposition of networks. *CoRR*, cs.DS/0310049, 2003.

[8] T. Ben-Nun, M. Sutton, S. Pai, and K. Pingali. Groute: An asynchronous multi-gpu programming model for irregular computations. In *PPoPP*, pages 235–248. ACM, 2017.

[9] C. Bron and J. Kerbosch. Finding all cliques of an undirected graph (algorithm 457). *Commun. ACM*, 16(9):575–576, 1973.

[10] L. Chang, M. Xu, and D. Strash. Efficient maximum k-plex computation over large sparse graphs. *Proc. VLDB Endow.*, 16(2):127–139, 2022.

[11] H. Chen, M. Liu, Y. Zhao, X. Yan, D. Yan, and J. Cheng. G-miner: an efficient task-oriented graph mining system. In *EuroSys*, pages 32:1–32:12. ACM, 2018.

[12] J. Chen, S. Cai, S. Pan, Y. Wang, Q. Lin, M. Zhao, and M. Yin. Nuqclq: An effective local search algorithm for maximum quasi-clique problem. In *AAAI*, pages 12258–12266. AAAI Press, 2021.

[13] J. Chen and X. Qian. Khuzdul: Efficient and scalable distributed graph pattern mining engine. In T. M. Aamodt, N. D. E. Jerger, and M. M. Swift, editors, *ASPLOS*, pages 413–426. ACM, 2023.

[14] X. Chen, R. Dathathri, G. Gill, and K. Pingali. Pangolin: An efficient and flexible graph mining system on CPU and GPU. *Proc. VLDB Endow.*, 13(8):1190–1205, 2020.

[15] Q. Cheng, D. Yan, T. Wu, L. Yuan, J. Cheng, Z. Huang, and Y. Zhou. Efficient enumeration of large maximal k-plexes. In A. Simitsis, B. Kemme, A. Queralt, O. Romero, and P. Jovanovic, editors, *EDBT*, pages 53–65. OpenProceedings.org, 2025.

[16] A. Ching, S. Edunov, M. Kabiljo, D. Logothetis, and S. Muthukrishnan. One trillion edges: Graph processing at facebook-scale. *Proc. VLDB Endow.*, 8(12):1804–1815, 2015.

[17] P. Cui, H. Liu, B. Tang, and Y. Yuan. Cggraph: An ultra-fast graph processing system on modern commodity CPU-GPU co-processor. *Proc. VLDB Endow.*, 17(6):1405–1417, 2024.

[18] Q. Dai, R. Li, H. Qin, M. Liao, and G. Wang. Scaling up maximal k-plex enumeration. In M. A. Hasan and L. Xiong, editors, *CIKM*, pages 345–354. ACM, 2022.

[19] V. V. dos Santos Dias, C. H. C. Teixeira, D. O. Guedes, W. M. Jr., and S. Parthasarathy. Fractal: A general-purpose graph pattern mining system. In *SIGMOD*, pages 1357–1374. ACM, 2019.

[20] Z. Fu, B. B. Thompson, and M. Personick. Mapgraph: A high level API for fast development of high performance graph analytics on gpus. In *Second International Workshop on Graph Data Management Experiences and Systems, GRADES 2014, co-loated with SIGMOD/PODS 2014, Snowbird, Utah, USA, June 22, 2014*, pages 2:1–2:6. CWI/ACM, 2014.

[21] S. Gao, K. Yu, S. Liu, and C. Long. Maximum k-plex search: An alternated reduction-and-bound method. *Proc. VLDB Endow.*, 18(2):363–376, 2024.

[22] J. E. Gonzalez, Y. Low, H. Gu, D. Bickson, and C. Guestrin. Powergraph: Distributed graph-parallel computation on natural graphs. In C. Thekkath and A. Vahdat, editors, *OSDI*, pages 17–30. USENIX Association, 2012.

[23] G. Guo, D. Yan, M. T. Özsu, Z. Jiang, and J. Khalil. Scalable mining of maximal quasi-cliques: An algorithm-system codesign approach. *Proc. VLDB Endow.*, 14(4):573–585, 2020.

[24] G. Guo, D. Yan, L. Yuan, J. Khalil, C. Long, Z. Jiang, and Y. Zhou. Maximal directed quasi-clique mining. In *ICDE*, pages 1900–1913. IEEE, 2022.

[25] L. Hu, L. Zou, and M. T. Özsu. Gamma: A graph pattern mining framework for large graphs on gpu. In *ICDE*. IEEE, 2023.

[26] K. Jamshidi, R. Mahadasa, and K. Vora. Peregrine: a pattern-aware graph mining system. In *EuroSys*, pages 13:1–13:16. ACM, 2020.

[27] J. Khalil, D. Yan, G. Guo, and L. Yuan. Parallel mining of large maximal quasi-cliques. *VLDB J.*, 31(4):649–674, 2022.

[28] F. Khorasani, K. Vora, R. Gupta, and L. N. Bhuyan. Cusha: vertex-centric graph processing on gpus. In B. Plale, M. Ripeanu, F. Cappello, and D. Xu, editors, *HPDC*, pages 239–252. ACM, 2014.

[29] G. Liu and L. Wong. Effective pruning techniques for mining quasi-cliques. In *PKDD*, volume 5212 of *Lecture Notes in Computer Science*, pages 33–49. Springer, 2008.

[30] Y. Liu, H. Huang, K. Yu, S. Liu, C. Long, and Z. Gu. Efficient size-bounded community search, revisited: Frameworks for practical improvements. *Proc. ACM Manag. Data*, 3(6):1–26, 2025.

[31] Y. Lu, J. Cheng, D. Yan, and H. Wu. Large-scale distributed graph computing systems: An experimental evaluation. *Proc. VLDB Endow.*, 8(3):281–292, 2014.

[32] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In *SIGMOD Conference*, pages 135–146. ACM, 2014.

[33] A. Maratea, L. Marcellino, and V. Duraccio. A gpu-parallel algorithm for fast hybrid BFS-DFS graph traversal. In *SITIS*, pages 450–457. IEEE Computer Society, 2017.

[34] K. Meng, J. Li, G. Tan, and N. Sun. A pattern based algorithmic autotuner for graph processing on gpus. In *PPoPP*, pages 201–213. ACM, 2019.

[35] X. Shi, X. Luo, J. Liang, P. Zhao, S. Di, B. He, and H. Jin. Frog: Asynchronous graph processing on GPU with hybrid coloring model. *IEEE Trans. Knowl. Data Eng.*, 30(1):29–42, 2018.

[36] S. Sun and Q. Luo. In-memory subgraph matching: An in-depth study. In *SIGMOD*, pages 1083–1098, 2020.

[37] X. Sun and Q. Luo. Efficient gpu-accelerated subgraph matching. *Proc. ACM Manag. Data*, 1(2):181:1–181:26, 2023.

[38] Y. Sun, J. Zhang, H. Cao, Y. Zhang, X. An, J. Huang, and X. Ye. Cgcgraph: Efficient CPU-GPU co-execution for concurrent dynamic graph processing. *ACM Trans. Archit. Code Optim.*, 22(3):91:1–91:26, 2025.

[39] C. H. C. Teixeira, A. J. Fonseca, M. Serafini, G. Siganos, M. J. Zaki, and A. Aboulnaga. Arabesque: a system for distributed graph mining. In *SOSP*, pages 425–440. ACM, 2015.

[40] J. R. Ullmann. An algorithm for subgraph isomorphism. *J. ACM*, 23(1):31–42, 1976.

[41] K. Wang, K. Yu, and C. Long. Maximal clique enumeration with hybrid branching and early termination. In *ICDE*, pages 599–612. IEEE, 2025.

[42] K. Wang, Z. Zuo, J. Thorpe, T. Q. Nguyen, and G. H. Xu. Rstream: Marrying relational algebra with streaming for efficient graph mining on a single machine. In *OSDI*, pages 763–782. USENIX Association, 2018.

[43] Y. Wang, A. A. Davidson, Y. Pan, Y. Wu, A. Riffel, and J. D. Owens. Gunrock: a high-performance graph processing library on the GPU. In R. Asenjo and T. Harris, editors, *PPoPP*, pages 11:1–11:12. ACM, 2016.

[44] Z. Wang, Y. Zhou, M. Xiao, and B. Khoussainov. Listing maximal k-plexes in large real-world graphs. In F. Laforest, R. Troncy, E. Simperl, D. Agarwal, A. Gionis, I. Herman, and L. Médini, editors, *WWW*, pages 1517–1527. ACM, 2022.

[45] Y. Wei and P. Jiang. Stmatch: Accelerating graph pattern matching on GPU with stack-based loop optimizations. In F. Wolf, S. Shende, C. Culhane, S. R. Alam, and H. Jagode, editors, *SC*, pages 53:1–53:13. IEEE, 2022.

[46] L. Xiang, A. Khan, E. Serra, M. Halappanavar, and A. Sukumaran-Rajam. cuts: scaling subgraph isomorphism on distributed multi-gpu systems using trie based data structure. In *SC*, pages 69:1–69:14. ACM, 2021.

[47] D. Yan, Y. Bu, Y. Tian, and A. Deshpande. Big graph analytics platforms. *Found. Trends Databases*, 7(1-2):1–195, 2017.

[48] D. Yan, J. Cheng, K. Xing, Y. Lu, W. Ng, and Y. Bu. Pregel algorithms for graph connectivity problems with performance guarantees. *Proc. VLDB Endow.*, 7(14):1821–1832, 2014.

[49] D. Yan, G. Guo, M. M. R. Chowdhury, M. T. Özsu, W. Ku, and J. C. S. Lui. G-thinker: A distributed framework for mining subgraphs in a big graph. In *ICDE*, pages 1369–1380. IEEE, 2020.

[50] D. Yan, G. Guo, J. Khalil, M. T. Özsu, W. Ku, and J. C. S. Lui. G-thinker: a general distributed framework for finding qualified subgraphs in a big graph with load balancing. *VLDB J.*, 31(2):287–320, 2022.

[51] D. Yan, L. Yuan, A. Ahmad, C. Zheng, H. Chen, and J. Cheng. Systems for scalable graph analytics and machine learning: Trends and methods. In R. Baeza-Yates and F. Bonchi, editors, *KDD*, pages 6627–6632. ACM, 2024.

[52] K. Yao and L. Chang. Efficient size-bounded community search over large networks. *Proc. VLDB Endow.*, 14(8):1441–1453, 2021.

[53] C. Ye, Y. Li, S. Sun, and W. Guo. gsword: Gpu-accelerated sampling for subgraph counting. *Proc. ACM Manag. Data*, 2(1):33:1–33:26, 2024.

[54] L. Yuan, A. Ahmad, D. Yan, J. Han, S. Adhikari, X. Yu, and Y. Zhou. G^2-aimd: A memory-efficient subgraph-centric framework for efficient subgraph finding on gpus. In *ICDE*, pages 3164–3177. IEEE, 2024.

[55] L. Yuan, G. Guo, D. Yan, S. Adhikari, J. Khalil, C. Long, and L. Zou. G-thinkerq: A general subgraph querying system with a unified task-based programming model. *IEEE Trans. Knowl. Data Eng.*, 37(6):3429–3444, 2025.

[56] L. Yuan, D. Yan, J. Han, A. Ahmad, Y. Zhou, and Z. Jiang. Faster depth-first subgraph matching on gpus. In *ICDE*, pages 3151–3163. IEEE, 2024.

[57] J. Zhong and B. He. Medusa: Simplified graph processing on gpus. *IEEE Trans. Parallel Distributed Syst.*, 25(6):1543–1552, 2014.

Algorithm 3 Ullmann's Subgraph Matching Algorithm

```
1: Input: query graph G_q = (V_q, E_q), data graph G = (V, E)
2: generate matching order π = [u_1, u_2, ..., u_{|V_q|}]   (u_i ∈ V_q)
3: ENUMERATE(∅, 1, π)
4: procedure ENUMERATE(S, i, π)
5:     C_S(u_i) ← viable vertex candidates in G to match u_i
6:     for all v ∈ C_S(u_i) do
7:         append S with (u_i, v)
8:         if |S| = k then output S
9:         else ENUMERATE(S, i + 1, π)
10:        pop (u_i, v) from S
```

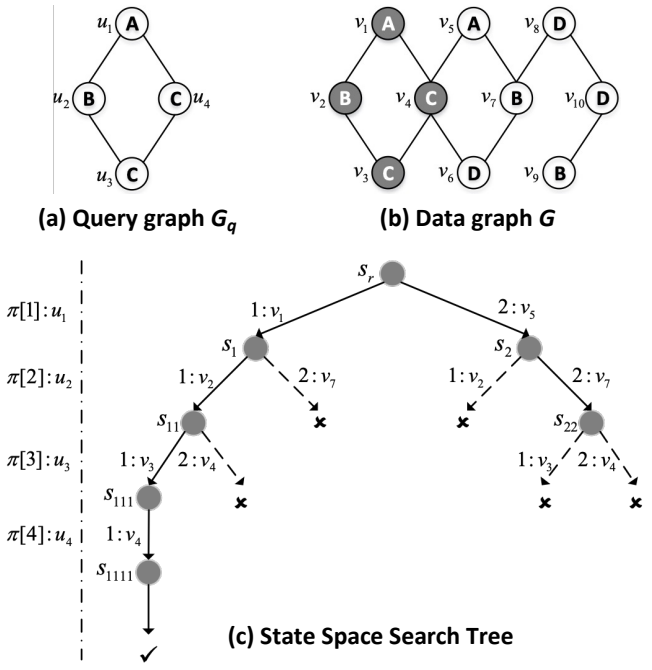


Figure 9: Illustration of Subgraph Matching

A Subgraph Matching

Subgraph matching (SM) is a representative SF problem that enumerates all subgraphs of a data graph G that are isomorphic to a query graph G_q. Many SM algorithms [**?** **?**] are based on Ullmann's backtracking framework [**?**], which orders query vertices and recursively extends a partial mapping while pruning infeasible candidates.

For completeness, we outline a basic Ullmann-style backtracking procedure in Algorithm **??**. The algorithm incrementally matches data vertices to query vertices $u_1, \ldots, u_{|V_q|}$ according to a predefined matching order, maintaining the current partial mapping in S. At each recursion level, viable candidate vertices in G are derived for the next query vertex, and infeasible extensions are pruned early based on adjacency constraints. The search proceeds recursively until either a complete match is found or no further extension is possible.

This process explores a state-space tree as illustrated in Figure **??**. A node at level i corresponds to a partial mapping that matches the query vertex prefix $[u_1, \ldots, u_i]$. For example, the partial mapping $S = [(u_1, v_5), (u_2, v_7), (u_3, v_4)]$ is pruned due to the absence of an edge (v_7, v_4) corresponding to the query edge (u_2, u_3). Various optimizations adopted by modern SM algorithms [**?**]—such as improving the matching order (Line **??**) and tightening candidate sets (Line **??**)—aim to reduce the search space but preserve this fundamental backtracking structure.

Importantly, the state-space tree explored by Ullmann-style algorithms is also a subgraph enumeration tree over G: each node represents a subgraph in G that matches a prefix of the query graph. Since the programming model of G-thinkerCG is built upon the subgraph enumeration tree and subgraph-task abstractions, it can naturally support CPU-GPU co-processing for SM algorithms by adapting their serial backtracking counterparts to the G-thinkerCG interface.

Table 3: Effect of System Parameters

Parameter	Value	soc-sinaweibo	wikipedia_link_sr
η / #{warps}	100	203.8	OOT
	500	113.9	542.3
	1000	**109.6**	**465.2**
	2000	231.0	948.0
CPU Chunk	1	117.1	357.6
	10	108.9	**316.1**
	50	**91.6**	465.2
	100	110.8	564.9
GPU Chunk	100K	143.3	511.9
	500K	**112.1**	**315.9**
	1M	159.4	1085.5
τ_c	1	220.3	567.3
	n_c / 2	**111.4**	**471.9**
	n_c / 4	112.5	476.7

B The Algorithm of `atomicSubNonNegative(.)`

To pop a subgraph from the stack $\mathcal{H}$, each thread of a warp calls the `pop_offsets(sglen)` operation shown in Figure **??**-❺, which properly decrements `otail` in Line 2, and returns the trackers `[offsets[ot-2], ohead[ot-1])` for all threads of the warp to read the obtained subgraph from $\mathcal{H}$.`vertices`. Note that since we cannot decrement `otail` to be smaller than 0, we cannot use `atomicSub(.)` since when multiple warps subtract 2 from `otail` consecutively, `otail` can become negative. We solve this issue by calling `atomicSubNonNegative(.)` as shown in Figure **??**-❻.

Specifically, Line 1 reads the value of `otail` into variable `old`, which is then read into `assumed` in Line 3. If this value is already 0 (Line 4), `otail` cannot be further decremented so Line 5 returns 0 directly as the original value of `otail`. Otherwise, Line 6 atomically checks if `otail`'s previously retrieved value (i.e., `old`) still equals `assumed`: (1) if so, no other warp has changed `otail` since Line 3, so the decremented value (`assumed−dec`) is written to `otail`; (2) while otherwise, `otail` has been updated by another warp, so we cannot update `otail` due to the atomicity requirement of subtraction, and `old` gets the latest value of `otail` in Line 6 and tries again until an atomic subtraction is successfully done (Line 7), after which the old value of `otail` before subtraction is returned in Line 8.

C System Parameter Study

We next present the effect of various system parameters on the running time of Hybrid using two representative datasets *soc-sinaweibo*

and *wikipedia_link_sr*. As shown in Table **??** (where OOT means running out of the maximum allowed time threshold of 1800 seconds), our default parameters generally achieve the best performance. Note that a tradeoff exists for each parameter: when η and vertex chunk sizes are too large, the task granularity is too coarse which hampers load balancing; while when η and GPU chunk size are too small, the computing power of GPU threads cannot be saturated, and when CPU chunk size is too small, the cost for CPU workers to wait in $\mathbb{W}$ increases. As for τ_c, if it is too small, GPU-to-CPU task stealing would be too aggressive and could lead to task thrashing if some CPU worker encounters a task timeout and adds many new tasks back to $\mathcal{S}$; while if it is too large, surplus GPU tasks cannot be timely rescheduled to CPU workers for processing. Our default parameters achieve a good tradeoff.